C PROGRAMMING II

Made By : **KASHIF SAYYAD**

MODULE I — FUNCTIONS

■ Introduction to Functions and their Types

A **function** is a self-contained block of program code which performs a well-defined task and which can be called whenever that task is needed. Programs are divided into functions so that the code becomes modular, easy to test, debug and reuse, because a task written once as a function can be called from anywhere many times.

Types of functions: library (built-in) functions like printf(), scanf(), sqrt(), strlen() which the compiler suppliers with the system; and **user-defined functions** which the programmer creates with the help of a name, a return type, parameters and a body. A user-defined function is called by the main() function or by any other function. **Function prototype** is only the declaration of the function — its name, return type and number / types of parameters — placed above main() so the compiler can check every call.

■ Defining Functions, Arguments, Actual and Formal Parameters

The **definition** of a function has a **return type**, a **name**, a **parameter list (formal parameters)** and a **body** ÐºÑ€ÑƒÑ‚'s block of statements ending with the return statement (for a non-void return type). The values actually passed at the time of the call are the **actual parameters (arguments)** and the names that receive them inside the function definition are the **formal parameters**.

```
void hello(void);                    /* prototype                      */
int  square(int x);                   /* one parameter x                */

int main(void)
{
    hello();
    printf("%d\n", square(5));  /* 5 = actual parameter   */
    return 0;
}

void hello(void)                       /* definition                  */
{
    printf("Hello!\n");
}

int square(int x)                      /* x = formal parameter        */
{
    return x * x;                      /* returning one result        */
}
```

■ Calling Function, Returning Function Results, Recursion

The **calling function** starts the execution of a function only at the time of the call with its arguments; the function runs, and after it finishes the execution control returns to the statement next after the call. A function returns back only one value to the caller, of the type given in its return type (return expression;); a void function returns nothing.

Recursion is the technique where a function calls itself, either directly or through another function, until some base (terminating) condition is met. Every recursive call stores its own copy of the automatic variables and parameters on the **stack**. The classic example is factorial: f(n) = n × f(n−1) with base f(0) = 1; also the Fibonacci series, towers of Hanoi, sum of n natural numbers and binary search with recursion. Recursion produces simple-looking code but repeated calls consume computer stack memory and may be slower than their iterative equivalents.

MODULE II — STRUCTURE, UNION AND POINTERS

■ Structure — Introduction, Declaration and Initialising

A **structure** is a user-defined **derived data type** that groups together variables of **different data types** under one name, so the whole group can be treated as one object (a record). Its declaration does not reserve memory; it only describes a layout: struct student { int roll; char name[30]; float marks; }; after defining the structure type, variables can be created: struct student s1; or many at once: struct student s[50];

Initialising structure at declaration: struct student s1 = {1, "ALI", 88.5f}; or per-field: s1.roll = 1; strcpy(s1.name, "ALI"); s1.marks = 88.5; When the structure is copied from one variable to another, member by member copying happens (struct variables can be assigned as a whole in C).

■ Accessing Structure Members, Nested Structures

Members of a structure variable are accessed with the **dot (.) member operator** — s1.roll, s1.marks. When the structure is reached through a pointer, the **arrow (->) operator** is used — ptr->roll is the same as (*ptr).roll. **Nested structures** means a structure that contains another complete structure as one of its members (employee contains a nested structure date for joining date); the inner member is reached by a.chain of dot operators, e.e1.dob.day. Similarly, a **structure array** keeps many records of the same structure type, accessed as st[i].marks.

■ typedef Statement and Enumerated Data Types

The **typedef statement** gives a new (shorter) name to an existing type. Typedef struct student STUDENT; after which STUDENT is used in place of struct student everywhere. It is not creating a new type; it is only a synonym, but it makes the declaration of the structure and pointer variables shorter and more readable.

An **enumerated (enum) data type** is a user-defined type whose values are a set of named integer constants listed by the programmer: enum days {SUN, MON, TUE, WED, THU=7, FRI, SAT}; The compiler gives value

0, 1, 2... by default (but after THU = 7, FRI becomes 8); the symbolic names make the program readable and the values can be compared and assigned like integers.

■ Unions, Declaration, Difference between Structure and Union

A **union** looks exactly like a structure in its declaration (union data { int i; float f; char c; };), but all its members **share one and the same memory** which starts at the same address. Hence writing to one member overwrites the value of the previous member, only one member is active at a time, and the size of the union equals the size of its largest member (here sizeof(int) < sizeof(float), so 4 bytes). A **structure** keeps **separate memory for every member**, all the members are active at once and total size is the sum, padding rounded to the alignment of the largest member.

Basis	**Structure**	**Union**
Memory of members	separate memory for every member	all members share one memory (same address)
Number of active members	all members at a time	only one member at a time
Size	sum of members (with padding)	size of the largest member
Keyword	struct	union

■ Pointers, The Address (&) and Indirection (*) Operators

A **pointer** is a variable that stores the **memory address** of another variable. The **address-of operator &** takes the address of a variable, and the **indirection (or dereference) operator *** takes the value present at that address. Both **declaration** and **initialisation** of pointers are done as int a = 10; int *p = &a; after which *p is 10, and p itself is the address of a.

A pointer can point to **functions**, **arrays** and **structures** and can also be the parameter of a function (call by reference). **Pointer expression and pointer arithmetic**: when p + 1 is applied to a pointer of type int, it moves the address by sizeof(int) (2 or 4 bytes), not really by 1; p + 1, p + 2, p−−, pointer − pointer (number of elements between), pointer comparisons and assignment of a null pointer are the permitted pointer operations. Multiplying or adding two pointers together is not permitted, and dereferencing is not allowed before checking for pointer validity.

Pointer to pointer stores the address of another pointer variable: int a = 5; int *p = &a; int **q = &p; then (*q) is p, and since *p is a, (**q) is 5. Dynamic memory of a pointer to pointer is also possible; levels of nesting are also possible.

■ Dynamic Memory Allocation in C

A normal array must have a fixed compile-time size; **dynamic memory allocation** requests memory **from the heap** at runtime of the required size. The four functions, all declared in **stdlib.h**, are:

- **malloc(size)** — allocates a block of size bytes and returns a void pointer to it; the memory is **not initialised** (contains garbage).
- **calloc(n, size)** — allocates n blocks of size each and **initialises every byte with 0**.
-
- **realloc(ptr, newsize)** — changes (grows or shrinks) the size of a previously allocated block and copies the old data into the new block.
-
- **free(ptr)** — releases the allocated block back to the heap; forgetting to call free creates a **memory leak**, and using a pointer after free is illegal.

MODULE III — STORAGE CLASSES, PREPROCESSORS AND FILE HANDLING

■ Storage Classes, Classes Scope, Visibility and Lifetime of Variables

A **storage class** tells the compiler about the **scope** (where the variable can be used), the **visibility** over other files, and **lifetime** (how long the variable stays alive in memory). The scope may be **block scope** (inside a pair of braces) or **file scope** (all the file, outside every function). The four storage classes of C are:

Storage class	Declared	Scope / visibility	Lifetime and default value
auto	inside a block	local to block only	lifetime = that block; garbage default
extern	outside all functions (also in other files)	visible to all files in the whole program	lifetime = whole program; initialised to 0
static	inside or outside a block	local to that file (or block)	memory and value retained between calls; 0 default
register	inside a block	local to block	requests CPU register storage; faster access, no address operator allowed

---

■ Preprocessor Directives

The **preprocessor** is a program that takes the C source text and rewrites it **before** actual compilation, on the commands written with # at the start of their lines:

- **File inclusion**: #include <header.h> (standard files) or #include "myfile.h" (user files) — the content of that file is pasted right there.
- **Macro substitution**: #define PI 3.14 replaces every PI in the program text by 3.14 before compilation; function-like macros like #define SQR(x) ((x)*(x)) take parameters; an #undef ends the macro.
- **Conditional compiler directives**: #if, #ifdef, #ifndef, #else, #elif, #endif — compile or skip parts conditionally; #error for user-defined errors; #line to reset line numbers.

A conditional compilation is useful for debugging and for code that works on more than one machine: when the condition inside the directives is true the code is compiled, otherwise it is skipped.

```
#define PI 3.14159
#define AREA(r)  (PI * (r) * (r))

int main(void)
{
#ifdef DEBUG
    printf("Debug mode ON\n");
#endif
    printf("Area = %.2f\n", AREA(2.0));
    return 0;
}
```

■ File Handling — Introduction, Opening and Closing a File

All file operations happen through a special pointer type **FILE *** (declared in stdio.h) and the **fopen()** function; fopen("file.txt", "r") returns the address of the file control block (NULL in case of failure), and after the work is over the **fclose(fp)** function closes the file and frees the buffer.

Mode	Purpose	If file does not exist	If file already exists
"r"	read only	returns NULL	reads from the beginning
"w"	write only	creates new	overwrites contents
"a"	append; add at end	creates new	keeps old data, writes at end
"r+"	read + write	returns NULL	read/write from beginning
"w+"	write + read	creates new	old data lost
"a+"	append + read	creates new	writes at the end of file

■ Input / Output Operations on Files

Text files hold readable characters; **binary files** hold the raw machine values. Getting and putting one character: **getc(fp)** reads one character, **putc(ch, fp)** writes one character. Formatted text: **fprintf(fp, "%d %s", ...)** and **fscanf(fp, "%d %s", ...)** work like printf / scanf but on the file. **Records in binary form**: **fread(&record, sizeof(rec), 1, fp)** and **fwrite(&record, sizeof(rec), 1, fp)** read or write whole records (e.g. a full struct of one employee) in one call.

After the record is written, the **file** should be carefully **kept closed**; writing and reading of the same record from a text file or from a binary file is then possible by record position; **appending, modifying and deleting a record** are performed with fopen(a-add), the record to be modified is copied, changed, written into a temporary file both or via reading again and rewriting at the same place.

Remember : Random access means to read or write a record in the middle without reading through all the earlier records. **fseek(fp, offset, direction)** with direction set to **SEEK_SET** (from beginning), **SEEK_CUR** (from current position), **SEEK_END** (from end) moves the file position marker directly to any point in the file; **ftell(fp)** reports the current byte position, **rewind(fp)** returns to the beginning, **flushall()** clears all buffers, and the pair of functions **remove("file")** and **rename("old", "new")** delete or rename a file from the disk.